A DEVELOPER'S DISCOVERY

# My AI Agent Can Finally See Inside the System It Builds.

OpenTelemetry traces + an MCP control plane for the AI agent building my Rust product.

OpenTelemetry | MCP | Rust | AI Agents | Control Plane

Lim Chun Zhe

THE PROBLEM

# The Agent Builds It. But Can't See It.

I'm building an organizational memory system in Rust. An AI agent (Claude Code) writes the code, runs the tests, ships the commits. But when something is slow or broken, it's flying blind.

WHAT THE AGENT COULDN'T ANSWER

❌ **"Why is search taking 120ms?"**
Is it embedding? Vector search? Both? No way to tell without manual profiling.

❌ **"Are there stale sessions accumulating?"**
Agent sessions can go orphaned. No visibility into lifecycle state.

❌ **"How many entries are in the knowledge store?"**
The agent writes to the system constantly. No dashboard, no counters.

The agent is both the builder and the first user. If it can't observe the system, it's debugging with println.

STEP 1: OBSERVABILITY

# OTEL Traces Across Every Operation

30 span constants across 7 subsystems. Every kernel operation instrumented at design time. Not bolted on after something broke.

```
// corvia-telemetry span contracts

pub const RAG_CONTEXT: &str = "corvia.rag.context";
pub const ENTRY_EMBED: &str = "corvia.entry.embed";
pub const STORE_SEARCH: &str = "corvia.store.search";
pub const MERGE_PROCESS: &str = "corvia.merge.process";
pub const GC_RUN: &str = "corvia.gc.run";

// ... 25 more across agent, session, inference, hooks
```

| 30 | 7 | p50-99 |
|---|---|---|
| Span constants | Subsystems traced | Latency percentiles |

REAL DATA

# From a Live Session

Real trace data from the running system. Every search, every embedding, every store operation. The agent reads these to understand performance.

```
GET /api/dashboard/traces

corvia.store.get 424 calls p50: 0.65ms p99: 1.10ms
corvia.store.search 23 calls p50: 26ms p95: 115ms
corvia.rag.context 23 calls p50: 42ms p95: 135ms
corvia.entry.embed 25 calls p50: 14ms p95: 36ms
corvia.entry.write 2 calls avg: 88ms
```

```
GET /api/dashboard/traces/recent (trace tree)

corvia.rag.context 42ms total, 3 spans
  └─ corvia.entry.embed 14ms
  └─ corvia.store.search 26ms ← 60-85% of time
```

Retrieval consistently takes 60-85% of RAG pipeline time. That insight redirected optimization from the embedding model to the vector search path.

THE NEXT QUESTION

# It Can See.
# But Can It Act?

Observability showed the agent what was happening.
But it had no way to change anything without running shell commands.

WHAT I WANTED TO DO BUT COULDN'T

📊 **Tune RAG parameters live**
Change top_k, similarity cutoff, graph expansion depth. Hot-reload. See the impact on the next query.

🔧 **Rebuild the vector index**
After a schema change or chunking update. No SSH, no CLI. Tell the agent to do it.

🔄 **Toggle features and A/B test**
Enable skills, change chunking strategy, adjust merge thresholds. All hot-reloadable.

Observability without control is a dashboard with no buttons. I want to operate the system through the agent, with the same protocol it already speaks.

STEP 2: CONTROL PLANE

# 10 MCP Tools.
# 3 Safety Tiers.

Same MCP server. Same protocol the agent already uses for knowledge.
Now it can also manage the system. With graduated trust.

### TIER 1: AUTO-APPROVED (5 TOOLS)

`system_status` `config_get` `agents_list` `adapters_list` `merge_queue`

### TIER 2: CONFIRM FIRST (3 TOOLS)

`config_set` `gc_run` `rebuild_index`

### TIER 3: CONFIRM + DRY-RUN (2 TOOLS)

`agent_suspend` `merge_retry`

Built on MCP's existing primitives (tool annotations, _meta). Read freely. Mutate with a preview. Destructive actions need both a dry-run and a confirmation.

WHAT THIS LEADS TO

# The Product Becomes A Lab Bench.

Observe, hypothesize, change, measure. All through MCP.
All without leaving the conversation.

1 **Observe**
Traces show retrieval is 60-85% of RAG latency. Graph expansion adds significant cost.

2 **Hypothesize**
"What if I disable graph expansion and increase top_k instead?"

3 **Act**
`config_set rag.graph_expansion_depth = 0`. Hot-reload. No restart.

4 **Measure**
Run the same queries. Read the traces. Compare p50 and p95 before and after.

Same loop works for A/B testing skills, hot-swapping embedding models, changing chunking strategies. The agent is both the experimenter and the instrument.

KEY TAKEAWAYS

# What I've Learned So Far

💡 **Instrument at design time, not after the fire**

Span contracts defined before the code existed. Every new feature gets traces automatically.

👁 **Observability without control is a dashboard with no buttons**

The agent needs graduated permission to act on what it sees. Not just watch.

🔒 **Safety tiers are the agent equivalent of "are you sure?"**

Built on MCP's existing primitives. Read freely. Mutate with preview. Destructive ops need dry-run.

🌱 **I'm still early. The tools are shipped. The workflows are forming.**

MCP standardized tool access. Agent control planes are next. The patterns exist. Someone needs to make them plug-and-play.

---

**What's the most interesting thing you've let an AI agent change in a running system? Or are you still keeping the keys?**

Would love to hear what patterns others have landed on.